

一、单项选择题

1.B 2.C 3.D 4.B 5.B 6.D 7.C 8.D 9.B 10.D

(正常情况下冒泡排序和快速排序都可以选, 但如果两个同时出现就选快排, 有一说法是冒泡排序固定执行 $n-1$ 次, 而不设置 flag 标志)

11.B 12.C 13.B 14.B 15.D

(这个题有点争议, 不确定要不要选 C, 但是查找效率取决于三个因素: 散列函数, 处理冲突的方法, 装填因子)

二、判断题

1.× 2.× 3.× 4.× 5.× 6.× 7.×

(查找成功: $(n+1)/2$, 查找失败 $(n+1)$)

8.√ 9.× 10.× 11.× 12.√ 13.× 14.× 15.×

三、填空题

1. 深度优先

2. 队列

3. $O(\log_e)$ 、 稀疏

4. 出度、 入度

5. $O(n^2)$

6. 2^{i-1}

7. $n+1$

8. 冒泡排序、 快速排序

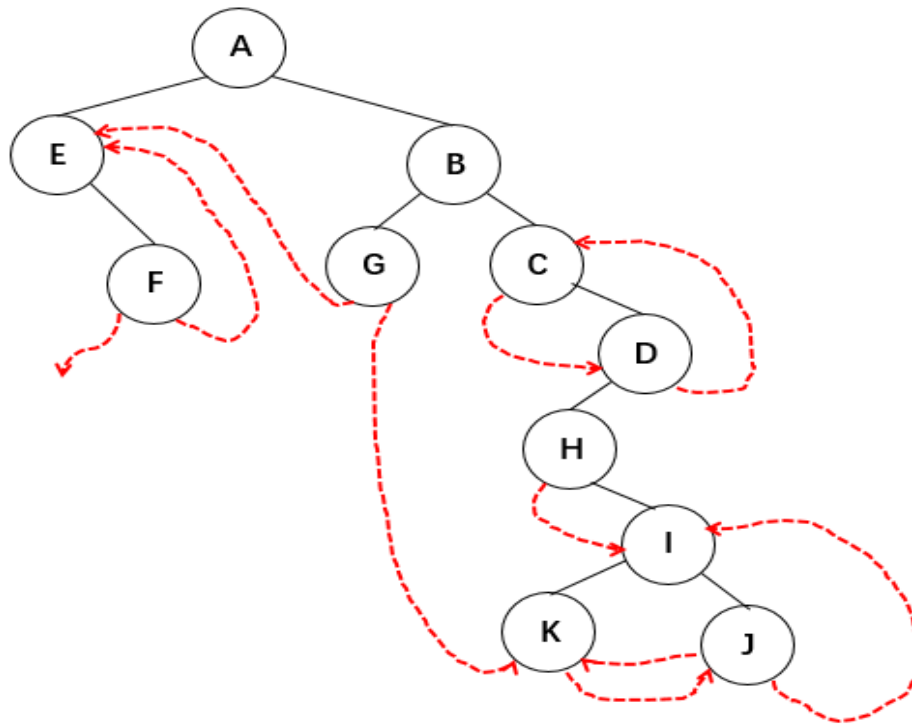
9. 前驱结点、 后继结点

10. 线性结构、 非线性结构

四、简答题

1. 由前序和中序遍历序列可唯一确定该二叉树，后序遍历序列为：

F E G K J I H D C B A，后序线索二叉树如下所示



2. 邻接矩阵

	A	B	C	D	E	F
A	0	1	1	∞	∞	∞
B	1	0	∞	1	∞	∞
C	1	∞	0	1	∞	∞
D	∞	1	1	0	1	∞
E	∞	∞	∞	1	0	1
F	∞	∞	∞	∞	1	0

深度优先：A B D C E F

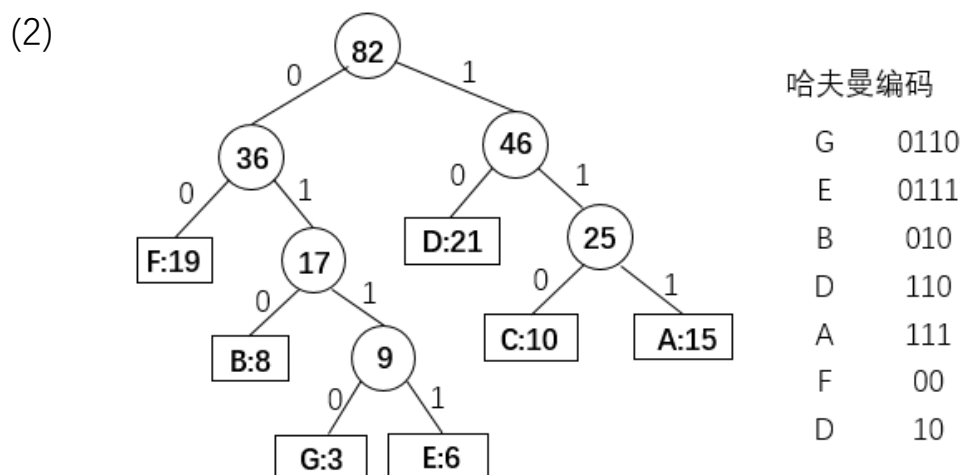
广度优先：A B C D E F

3. 带权邻接矩阵

	V ₀	V ₁	V ₂	V ₃	V ₄	V ₅
V ₀	0	30	30	20	∞	60
V ₁	∞	0	25	∞	∞	∞
V ₂	∞	∞	0	∞	∞	35
V ₃	∞	∞	∞	0	∞	50
V ₄	∞	∞	∞	∞	0	∞
V ₅	∞	∞	∞	∞	45	0

终点	从V0到各终点的dist的值和最短距离和路线				
V ₁	30 V ₀ →V ₁	30 V ₀ →V ₁			
V ₂	30 V ₀ →V ₂	30 V ₀ →V ₂	30 V ₀ →V ₂		
V ₃	20 V ₀ →V ₃				
V ₄	∞	∞	∞	∞	105 V ₀ →V ₅ →V ₄
V ₅	60 V ₀ →V ₅	60 V ₀ →V ₅	60 V ₀ →V ₅	60 V ₀ →V ₅	
V _j	{V ₀ ,V ₃ }	{V ₀ ,V ₃ ,V ₁ }	{V ₀ ,V ₃ ,V ₁ ,V ₂ }	{V ₀ ,V ₃ ,V ₁ ,V ₂ ,V ₅ }	{V ₀ ,V ₃ ,V ₁ ,V ₂ ,V ₅ ,V ₄ }

4 .(1) 算法思想: ①将 7 个权值分别看作只有根结点的 7 棵二叉树, 这些二叉树构成的集合记为 H。②从 H 中选出两棵根结点权值最小的树(假设为 a, b)作为左、右子树构建成为一个新的二叉树(命名为 T), 新的二叉树的根结点的权值为左、右子树根结点的权值之和。③从集合 H 中删除 a, b, 再加入新构造的树 T。④重复②③两步, 直至 H 中只剩下一棵树为止, 这棵树就是哈夫曼树。



【注】哈夫曼编码的先后顺序并无大碍, 看得整齐就好, 不用在意

(3) 哈夫曼编码采用可变长度编码方式, 对出现频率高的字符赋以短编码, 而对频率较低的字符则赋以较长一些的从而可以使字符平均编码长度减短, 起到压缩数据的效果。

5 .(1) hash 表是一种根据关键字而直接进行访问的数据结构。它借助于 hash 函数把查找表中的关键字映射成该关键字对应的地址，hash 表则是建立了关键字和存储地址之间的一种直接映射关系。

(2) . 由散列函数计算出各关键字对应的散列地址如下表所示：

关键字	55	21	18	27	11	45	33	26	27
地址	0	10	7	5	0	1	0	4	5

由此构造出以线性探测解决冲突的散列表为：

地址	0	1	2	3	4	5	6	7	8	9	10	11
关键字	55	11	45	33	26	27	27	18			21	

$$(3) ASL_{成功} = (1+2+2+4+1+1+2+1+1)/9 = 15/9$$

$$ASL_{失败} = (9+8+7+6+5+4+3+2+1+1+2+1)/11 = 49/11$$

五、算法设计题

1. 1)

```

1 void CreateGraph(MGraph &G)
2 {
3     int i, j, k, weight;
4     int Gn, Ge;
5     print("请输入顶点数和边数:\n");
6     scanf("%d %d", &Gn, &Ge);
7
8     print("请输入顶点:\n");
9     for(i = 0; i < Gn; i++)
10         scanf("%d", &vexs[i]);
11     for(i = 0; i < Ge; i++)
12     {
13         for(j=0; j < Gn; j++)
14             G.AdjMatrix[i][j] = infinity;
15     }
16 }

```

```

16
17     print("请输入边的顶点下标i,j以及权重w\n");
18     for(k=0; k < Ge; k++)
19     {
20         scanf("%d %d %d",&i,&j,&weight)
21         G.AdjMatrix[i][j] = weight;
22         G.AdjMatrix[j][i] = G.AdjMatrix[i][j];
23     }
24 }

```

2)

```

1  bool visited[MAX_VERTEX_NUM];    // 访问标记数组
2  void BFST(MGraph G)
3  {
4      for(i=0; i<Gn; ++i)
5          visited[i] = FALSE;      // 访问标记数组初始化
6
7      InitQueue(Q);                 // 初始化辅助队列
8
9      for(i=0; i<Gn; ++i)           // 从0号顶点开始遍历
10         if(!visited[i])            // 对每个 连通分量调用一次BFS
11             BFS(G, i);              // Vi未访问过, 从Vi开始BFS
12 }
13
14 void BFS(MGraph G, int v)          // 从顶点v出发, 广度优先遍历图G.
15 {
16     visit(v);                      // 访问初始顶点v
17     visited[v]=TRUE;                // 对v做已访问标记
18     EnQueue(Q, v);
19     while(!IsEmpty(Q))
20     {
21         DeQueue(Q,v);
22         // 检测v的所有邻接顶点
23         for(w=FirstNeighbor(G, v); w>=0; w=NextNeighbor(G,v,w))
24             if(!visited[w])         // w为v的尚未访问的邻接顶点
25             {
26                 visit(w);
27                 visited[w] = TRUE;
28                 EnQueue(Q,w);
29             }
30     }
31 }
32

```

2.

```

1 void link_ins(NodeType *head, datatype x, datatype elm)
2 {
3     NodeType *p=head, *s;
4     while(p->next != NULL)
5     {
6         if(p->next->data != elm)
7             p = p->next;
8         else
9             break;
10    }
11    s = (NodeType*)malloc(sizeof(NodeType));
12    s->data = x;
13    s->next = p->next;
14    p->next = s;
15 }

```

3. (1)

```

1 void Bubble_Sort(Sqlist &L)
2 {
3     int i, j, temp, flag;
4     for(i=L.length-1; i>=1; --i)
5     {
6         flag = 0; //变量flag用来标记本趟排序是否发生交换
7         for(j=1; j<=i; ++j)
8             if(L.r[j-1].key > L.r[j].key)
9             {
10                temp = L.r[j];
11                L.r[j] = L.r[j-1];
12                L.r[j-1] = temp;
13                //如果没发生交换, 则flag的值为0, 如果发生交换, 则flag的值为1
14                flag = 1;
15            }
16        //一趟排序过程中没有发生关键字的交换, 则证明序列有序, 排序结束
17        if(flag == 0) return;
18    }
19 }

```

(2)

```

1 int Binary_Search(Sqlist L, ElementType k)
2 {
3     //在有序表L中查找关键字为k的元素, 若存在
4     //则返回其位置, 不存在则返回 -1
5     int low=0, high=L.length-1, mid;
6     while(low <= high) //当前子表长度大于等于1 时进行循环
7     {
8         mid = (low + high)/2; //取中间位置
9         if(L.element[mid].key == k)
10            return mid; //查找成功则返回所在位置
11        else if(L.element[mid].key > k)
12            high = mid - 1; //从前半部分继续查找
13        else
14            low = mid + 1; //从后半部分继续查找
15    }
16
17    return - 1;
18 }

```